

**Mathematical Programming and
Operations Research**
Modeling, Algorithms, and Complexity
Examples in Excel and Python
(Work in progress)

Edited by: Robert Hildebrand

Contributors: Robert Hildebrand, Laurent Poirrier, Douglas Bish, Diego Moran

Version Compilation date: June 16, 2025

Preface

This entire book is a working manuscript. The first draft of the book is yet to be completed.

This book is being written and compiled using a number of open source materials. We will strive to properly cite all resources used and give references on where to find these resources. Although the material used in this book comes from a variety of licences, everything used here will be CC-BY-SA 4.0 compatible, and hence, the entire book will fall under a CC-BY-SA 4.0 license.

MAJOR ACKNOWLEDGEMENTS

I would like to acknowledge that substantial parts of this book were borrowed under a CC-BY-SA license. These substantial pieces include:

- "A First Course in Linear Algebra" by Lyryx Learning (based on original text by Ken Kuttler). A majority of their formatting was used along with selected sections that make up the appendix sections on linear algebra. We are extremely grateful to Lyryx for sharing their files with us. They do an amazing job compiling their books and the templates and formatting that we have borrowed here clearly took a lot of work to set up. Thank you for sharing all of this material to make structuring and forming this book much easier! See subsequent page for list of contributors.
- "Foundations of Applied Mathematics" with many contributors. See <https://github.com/Foundations-of-Applied-Mathematics>. Several sections from these notes were used along with some formatting. Some of this content has been edited or rearranged to suit the needs of this book. This content comes with some great references to code and nice formatting to present code within the book. See subsequent page with list of contributors.
- "Linear Inequalities and Linear Programming" by Kevin Cheung. See <https://github.com/dataopt/lineqlpbook>. These notes are posted on GitHub in a ".Rmd" format for nice reading online. This content was converted to $\text{\LaTeX}$ using Pandoc. These notes make up a substantial section of the Linear Programming part of this book.
- Linear Programming notes by Douglas Bish. These notes also make up a substantial section of the Linear Programming part of this book.

I would also like to acknowledge Laurent Porrier and Diego Moran for contributing various notes on linear and integer programming.

I would also like to thank Jamie Fravel for helping to edit this book and for contributing chapters, examples, and code.

Contents

Contents	5
1 Resources and Notation	5
2 Mathematical Programming	9
2.1 Warm-Up Scenario	9
2.2 Why Study Operations Research?	11
2.3 What is Mathematical Programming?	11
2.4 Applications	12
2.5 Types of Optimization problems	13
2.6 Linear Programming (LP)	13
2.7 Mixed-Integer Linear Programming (MILP)	14
2.8 Non-Linear Programming (NLP)	16
2.8.1 Convex Programming	17
2.8.2 Non-Convex Non-linear Programming	17
2.8.3 Machine Learning	18
2.9 Mixed Integer Non-Linear Programming (MINLP)	19
2.9.1 Convex MINLP	20
2.9.2 Non-Convex MINLP	20
2.10 Optimization Under Uncertainty	23
2.10.1 Stochastic Optimization	23
2.10.2 Robust Optimization	23
2.10.3 Applications	23
I Linear Programming	25
3 Modeling: Linear Programming	27
3.1 Introductory LP Example	28
3.2 Modeling and Assumptions in Linear Programming	29
3.2.1 Assumptions	31
3.3 Examples	32
3.3.1 Knapsack Problem	40
3.3.2 Capital Investment	41
3.3.3 Assignment Problem	42
3.4 Exercises	43
4 Software - Excel	45

4.1	Using Excel Solver	45
4.1.1	Useful Excel Functions and Techniques	45
4.1.2	How to Use Excel Solver	47
4.2	Exercises	51
5	Graphically Solving Linear Programs	53
5.1	Nonempty and Bounded Problem	53
5.2	Graphical Approach	55
5.3	Infinitely Many Optimal Solutions	59
5.4	Problems with No Solution	61
5.5	Problems with Unbounded Feasible Regions	62
5.6	Extreme Directions	68
5.7	Exercises	71
	Solutions	72
6	Modeling with Compact Notation	75
6.0.1	Notes to expand on	76
6.1	Production Planning Models	77
6.2	Assignment Problem	81
6.3	Assignment Problem	84
6.4	Modeling Tricks	91
6.4.1	Maximizing a minimum	91
6.5	Network Flow Models and Applications	92
6.5.1	Graphs	92
6.5.1.1	Undirected Graphs	92
6.5.1.2	Directed Graphs	93
6.5.2	Example: Minimum-Cost Flow in a Transportation Network	94
6.5.3	Minimum Cost Network Flow Problem	95
6.5.4	Maximum flow	97
6.6	Exercises	100
7	Formal Mathematical Statements	103
7.0.1	Vectors and Their Properties	103
7.0.2	Linear Combinations and Related Concepts	104
7.1	Convexity	105
7.1.1	Convex Sets	105
7.1.2	Spanning Sets and Basis	107
7.1.3	Convex and Polyhedral Sets	108
7.1.4	Unbounded Sets and Rays	111
8	Simplex Method	113
8.1	Standard Form	113
8.1.1	Standard Form for Constraints	115
8.1.2	Converting a Minimization Problem to a Maximization Problem	119

8.1.3	Exercises: Converting to Standard Form	120
8.2	Solutions to $A\mathbf{x} = \mathbf{b}$	121
8.2.1	Partitioning the Variables	123
8.2.2	Another Example	125
8.2.3	Selecting and Evaluating Different Bases	125
8.3	Optimality Conditions	126
8.4	Pivoting and the Simplex Algorithm	130
8.4.1	Transforming the LP to Standard Form	131
8.4.2	Summarizing the steps taken	137
8.5	The Simplex Algorithm!	138
8.6	No Feasible Initial Basis and the Big-M Method	142
8.6.1	Let's see another example	149
8.7	Degeneracy, Cycling, and Bland's Rule	151
8.7.1	Degeneracy and Unchanged BFS after Pivoting	151
8.7.2	Cycling in the Simplex Method	152
8.7.3	Bland's Rule	153
8.8	Simplex and Unbounded LPs	154
8.9	Exercises	156
9	Sensitivity Analysis	161
9.0.1	Mathematical details of sensitivity analysis	161
9.1	Sensitivity Analysis in the Revised Simplex Tableau	162
9.1.1	Effect of Changing an Objective Coefficient c_i	163
9.1.2	Effect of Changing the Right-Hand Side $\mathbf{b}$	163
9.1.3	Effect of Changing the Coefficients a_{ij} of the Constraint Matrix A	164
9.1.4	Summary Table: Sensitivity Effects in the Revised Simplex Tableau	165
9.1.5	Computing the Range of Parameters That Maintain the Optimal Basis	165
9.1.5.1	Original Problem and Optimal Tableau	165
9.1.5.2	1. Changing the Right-Hand Side b_1 of the First Constraint	166
9.1.5.3	Sensitivity Analysis on the Objective Coefficient c_x	167
9.1.6	Excel Solver Sensitivity Analysis	168
10	Duality	171
10.1	Duality Theory	171
10.2	Primal-Dual Pairs and Strong Duality	174
10.3	Different Primal–Dual Forms	176
10.4	Exercises	185
10.5	Complementary Slackness and Optimality Conditions	188
10.5.1	Definition of Complementary Slackness	188
10.5.2	Interpreting Complementary Slackness	189
10.5.3	Using Complementary Slackness to Solve Problems	190
10.5.4	Summary of Key Takeaways	190
10.6	Exercises	191

11 Multi-Objective Optimization	193
11.1 Introduction to Multi-Objective Optimization	193
11.1.1 Further Methods for Multi-Objective Optimization	196
11.2 Multi-Objective Optimization and the Pareto Frontier	196
11.2.1 Where Are Pareto Frontiers Useful?	200
11.2.1.1 Political Redistricting	201
11.2.1.2 Portfolio Optimization	202
11.2.1.3 Aircraft Design	202
11.2.1.4 Vehicle Dynamics	202
11.2.1.5 Sustainable Construction	202
11.3 Brief Introduction to pymoo	203
11.4 References	204
 II Discrete Algorithms	 205
12 Graph Algorithms	207
12.1 Graph Theory and Network Flows	207
12.2 Graphs	207
12.2.1 Drawing Graphs	207
12.3 Definitions	210
12.4 Shortest Path	213
12.5 Spanning Trees	224
12.5.1 Kruskal's Algorithm for Minimum Cost Spanning Tree	227
12.6 Exercise Answers	232
12.6.1 Prim's Algorithm for Minimum Cost Spanning Tree	232
12.7 Additional Exercises	234
12.7.1 Notes	242
 III Integer Programming	 245
13 Integer Programming Formulations	247
13.1 Knapsack Problem	247
13.2 Capital Budgeting	250
13.3 Capacitated Lot Sizing Problem (CLSP)	253
13.4 Facility Location	254
13.4.1 Capacitated Facility Location	254
13.4.2 Uncapacitated Facility Location	257
13.5 Set Covering	259
13.5.1 Covering (Generalizing Set Cover)	262
13.6 Graph Coloring	263
13.7 Basic Modeling Tricks - Using Binary Variables	265
13.7.1 Big M constraints - Activating/Deactivating Inequalities	267

13.7.2	Either Or Constraints	271
13.7.3	If then implications - opposite direction	272
13.7.4	Multi Term Disjunction with application to 2D packing	274
13.7.4.1	Strip Packing Problem	275
13.7.5	SOS1 Constraints	276
13.7.6	SOS2 Constraints	277
13.7.7	Piecewise linear functions with SOS2 constraint	278
13.7.7.1	SOS2 with binary variables	280
13.7.8	Maximizing a minimum	281
13.7.9	Relaxing (nonlinear) equality constraints	281
13.7.10	Exact absolute value	281
13.7.10.1	Exact 1 -norm	282
13.7.10.2	Maximum	282
13.8	Job Shop Scheduling	283
13.8.1	JSSP Components	283
13.8.2	Mathematical Model	285
13.8.3	Job Shop Scheduling Variations	287
13.9	Quadratic Assignment Problem (QAP)	288
13.10	Generalized Assignment Problem (GAP)	290
13.10.1	In special cases	290
13.10.2	Explanation of definition	290
13.11	Other material	291
13.11.1	Binary reformulation of integer variables	291
13.12	Literature and Resources	293
13.13	MIP Solvers and Modeling Tools	294
13.13.1	Tools for Solving Job Shop Scheduling Problems	294
14	Algorithms to Solve Integer Programs	297
14.1	Foundational Principle - LP is a relaxation for IP	298
14.1.1	Rounding LP Solution can be bad!	299
14.1.2	Rounding LP solution can be infeasible!	300
14.2	Branch and Bound	300
14.2.1	Binary Integer Programming	300
14.2.2	Branch and bound on general integer variables	302
14.3	Cutting Planes	305
14.3.1	Cover Inequalities	305
14.3.2	Chvátal Cuts	306
14.3.3	Cutting Planes Procedure	307
14.3.4	Gomory Cuts	310
14.4	Interpreting Output Information and Progress	313
14.5	Branching Decision Rules in Integer Programming	314
14.6	Decomposition Methods - Advanced approaches to integer programming	317
14.6.1	Lagrangian Relaxation	317
14.6.2	Dantzig-Wolfe Reformulation and Column Generation	318

14.6.3	Benders Decomposition	319
14.7	Literature and Resources	320
15	Exponential Size Formulations	321
15.1	Cutting Stock	321
15.1.1	Pattern formulation	324
15.1.2	Column Generation	326
15.1.3	Cutting Stock - Multiple widths	327
15.2	Traveling Salesman Problem	328
15.2.1	Miller Tucker Zemlin (MTZ) Model	330
15.2.2	Dantzig-Fulkerson-Johnson (DFJ) Model	337
15.2.3	Traveling Salesman Problem - Branching Solution	340
15.2.4	Traveling Salesman Problem Variants	340
15.2.4.1	Many salespersons (m-TSP)	340
15.2.4.2	TSP with order variants	342
15.2.5	Multi vehicle model with capacities	342
15.3	TSP with Time Windows	343
15.4	Case study - Traveling Salesman Problem Applications for Supply Chain Optimization	344
15.5	Vehicle Routing Problem (VRP)	347
15.5.1	The Clarke-Wright Saving Algorithm for VRP	348
15.5.2	Concept	348
15.5.3	Saving Calculation	348
15.5.4	Algorithm Steps	348
15.5.5	Advantages and Limitations	349
15.6	Tools to solve VRP	349
15.7	Literature and other notes	351
15.7.1	TSP In Excel	351
15.7.2	Resources	351
16	Algorithms and Complexity	353
16.1	Big-O Notation	353
16.2	Algorithms - Example with Bubble Sort	357
16.2.1	Sorting	357
16.3	Problem, instance, size	362
16.3.1	Problem, instance	362
16.3.2	Format and examples of problems/instances	362
16.3.3	Size of an instance	362
16.4	Complexity Classes	363
16.4.1	P	364
16.4.2	NP	365
16.4.3	Problem Reductions	366
16.4.4	Significance of Reductions	368
16.4.5	Examples of Problem Reductions	368
16.4.5.1	VERTEX COVER to SET COVER	368

16.4.5.2	3SAT to 3D MATCHING	368
16.4.5.3	Discussion	369
16.4.6	NP-Hard	369
16.4.7	NP-Complete	369
16.5	Problems and Algorithms	371
16.5.1	Matching Problem	371
16.5.1.1	Greedy Algorithm for Maximal Matching	372
16.5.1.2	Other algorithms to look at	373
16.5.2	Minimum Spanning Tree	373
16.5.3	Kruskal's algorithm	374
16.5.3.1	Prim's Algorithm	374
16.5.4	Traveling Salesman Problem	374
16.5.4.1	Nearest Neighbor - Construction Heuristic	374
16.5.4.2	Double Spanning Tree - 2-Apx	375
16.5.4.3	Christofides - Approximation Algorithm - $(3/2)$ -Apx	377
16.6	Resources	377
16.6.1	Advanced - NP Completeness Reductions	378
16.7	Other material for Integer Linear Programming	378
	Exercises	383
	Solutions	383
16.7.1	Other discrete problems	384
16.7.2	Assignment Problem and the Hungarian Algorithm	384
16.7.3	History of Computation in Combinatorial Optimization	384
17	Heuristics for TSP	385
17.1	Construction Heuristics	385
17.1.1	Random Solution	385
17.1.2	Nearest Neighbor	385
17.1.3	Insertion Method	386
17.2	Improvement Heuristics	386
17.2.1	2-Opt (Subtour Reversal)	386
17.2.2	3-Opt	388
17.2.3	k -Opt	388
17.3	Meta-Heuristics	388
17.3.1	Hill Climbing (2-Opt for TSP)	388
17.3.2	Simulated Annealing	390
17.3.3	Tabu Search	391
17.3.4	Genetic Algorithms	392
17.3.5	Greedy randomized adaptive search procedure (GRASP)	392
17.3.6	Ant Colony Optimization	392
17.4	Computational Comparisons	392
17.4.1	VRP - Clark Wright Algorithm	393
18	Forecasting and Stochastic Programming	395

19 Decomposition Methods	397
19.1 Lagrangian Relaxation	397
19.1.1 Motivation	397
19.1.2 Formulation	397
19.1.3 Solution	398
19.1.4 Optimizing Lagrangian Dual	399
19.1.5 Optimization	399
19.1.6 Termination	400
19.1.7 Conclusion	400
19.1.8 Motivation	401
19.1.9 Vehicle Routing Problem	402
19.1.10 Lagrangian Relaxation for VRP	402
19.1.11 Conclusion	403
19.1.12 Motivation	403
19.1.13 Vehicle Routing Problem	403
19.1.14 Lagrangian Relaxation for VRP	404
19.1.15 Conclusion	404
19.1.16 Motivation	404
19.1.17 Subtour Elimination Constraints	404
19.1.18 Lagrangian Relaxation	405
19.1.19 Conclusion	405
19.2 Embedding Lagrangian relaxation into a branch and bound tree	408
19.2.1 Motivation	408
19.2.2 Lagrangian Relaxation	408
19.2.3 Branch and Bound with Lagrangian Relaxation	409
19.2.4 Conclusion	409
19.2.5 Motivation	409
19.2.6 Lagrangian Relaxation	409
19.2.7 Branch and Bound with Lagrangian Relaxation	410
19.2.8 Conclusion	410
 IV Nonlinear Programming	 415
20 Nonlinear Programming (NLP)	417
20.1 Definitions and Theorems	417
20.1.1 Calculus: Derivatives	419
20.1.2 Calculus: Second derivatives	420
20.1.3 Multivariate Calculus Examples	421
20.1.4 Taylor's Theorem	423
20.1.5 Constrained Minimization and the KKT Conditions	425
 21 NLP Algorithms	 429
21.1 Algorithms Introduction	429

21.2	1-Dimensional Algorithms	429
21.2.1	Golden Search Method - Derivative Free Algorithm	432
21.2.1.1	Example:	433
21.2.2	Bisection Method - 1st Order Method (using Derivative)	433
21.2.2.1	Minimization Interpretation	433
21.2.2.2	Root finding Interpretation	434
21.2.3	Gradient Descent - 1st Order Method (using Derivative)	434
21.2.4	Newton's Method - 2nd Order Method (using Derivative and Hessian)	437
21.3	Multi-Variate Unconstrained Optimizaition	438
21.3.1	Descent Methods - Unconstrained Optimization - Gradient, Newton	438
21.3.1.1	Choice of α_t	439
21.3.1.2	Choice of d_t using $\nabla f(x)$	439
21.3.2	Stochastic Gradient Descent - The mother of all algorithms.	439
21.3.3	Neural Networks	441
21.3.4	Choice of Δ_k using the hessian $\nabla^2 f(x)$	441
21.4	Constrained Convex Nonlinear Programming	441
21.4.1	Barrier Method	442
22	Computational Issues with NLP	447
22.1	Irrational Solutions	447
22.2	Discrete Solutions	447
22.3	Convex NLP Harder than LP	447
22.4	NLP is harder than IP	448
22.5	Resources	449
23	Fairness in Algorithms	451
24	One-dimensional Optimization	453
25	Gradient Descent Methods	463
A	Linear Equations	469
A.1	Graphing a Linear Equation	469
A.2	Slope of a Line	473
A.3	Determining the Equation of a Line	475
A.4	Applications of Linear Equations	476
A.5	More Applications: Systems of Linear Equations	477
B	Systems of Equations	481
B.1	Systems of Equations, Geometry	481
B.2	Systems Of Equations, Algebraic Procedures	485
B.2.1	Elementary Operations	487
B.2.2	Gaussian Elimination	491
B.2.3	Uniqueness of the Reduced Row-Echelon Form	503
B.2.4	Rank and Homogeneous Systems	505

C	Matrices	511
C.1	Matrix Arithmetic	511
C.1.1	Addition of Matrices	513
C.1.2	Scalar Multiplication of Matrices	515
C.1.3	Multiplication of Matrices	516
C.1.4	The ij^{th} Entry of a Product	523
C.1.5	Properties of Matrix Multiplication	526
C.1.6	The Transpose	527
C.1.7	The Identity and Inverses	529
C.1.8	Finding the Inverse of a Matrix	531
D	$\mathbb{R}^n$	537
D.1	Vectors in $\mathbb{R}^n$	537
D.2	Algebra in $\mathbb{R}^n$	540
D.2.1	Addition of Vectors in $\mathbb{R}^n$	541
D.2.2	Scalar Multiplication of Vectors in $\mathbb{R}^n$	542
D.3	Geometric Meaning of Vector Addition	543
D.4	Length of a Vector	546
D.5	Geometric Meaning of Scalar Multiplication	550
D.6	The Dot Product	552
D.6.1	The Dot Product	552
D.6.2	The Geometric Significance of the Dot Product	555
D.7	Contributors	559
	Index	3

Introduction

Letter to instructors

This is an introductory book for students to learn optimization theory, tools, and applications. The two main goals are (1) students are able to apply the of optimization in their future work or research (2) students understand the concepts underlying optimization solver and use this knowledge to use solvers effectively.

This book was based on a sequence of course at Virginia Tech in the Industrial and Systems Engineering Department. The courses are *Deterministic Operations Research I* and *Deterministic Operations Reserach II*. The first course focuses on linear programming, while the second course covers integer programming an nonlinear programming. As such, the content in this book is meant to cover 2 or more full courses in optimization.

The book is designed to be read in a linear fashion. That said, many of the chapters are mostly independent and could be rearranged, removed, or shortened as desited. This is an open source textbook, so use it as you like. You are encouraged to share adaptations and improvements so that this work can evolve over time.

Letter to students

This book is designed to be a resource for students interested in learning what optimizaiton is, how it works, and how you can apply it in your future career. The main focus is being able to apply the techniques of optimization to problems using computer technology while understanding (at least at a high level) what the computer is doing and what you can claim about the output from the computer.

Quite importantly, keep in mind that when someone claims to have *optimized* a problem, we want to know what kind of gaurantees they have about how good their solution is. Far too often, the solution that is provided is suboptimal by 10%, 20%, or even more. This can mean spending excess amounts of money, time, or energy that could have been saved. And when problems are at a large scale, this can easily result in millions of dollars in savings.

For this reason, we will learn the perspective of *mathematical programming* (a.k.a. mathematical optimization). The key to this study is that we provide gaurantees on how good a solution is to a given problem. We will also study how difficult a problem is to solve. This will help us know (a) how long it might take to solve it and (b) how good a of a solution we might expect to be able to find in reasonable amount of time.

2 ■ CONTENTS

We will later study *heuristic* methods - these methods typically do not come with gaurantees, but tend to help find quality solutions.

Note: Although there is some computer programming required in this work, this is not a course on programming. Thanks to the fantastic modelling packages available these days, we are able to solve complicated problems with little programming effort. The key skill we will need to learn is *mathematical modeling*: converting words and ideas into numbers and variables in order to communicate problems to a computer so that it can solve a problem for you.

As a main element of this book, we would like to make the process of using code and software as easy as possible. Attached to most examples in the book, there will be links to code that implements and solves the problem using several different tools from Excel and Python. These examples can should make it easy to solve a similar problem with different data, or more generally, can serve as a basis for solving related problems with similar structure.

How to use this book

Skim ahead. We recommend that before you come across a topic in lecture, that you skim the relevant sections ahead of time to get a broad overview of what is to come. This may take only a fraction of the time that it may take for you to read it.

Read the expected outcomes. At the beginning of each section, there will be a list of expected outcomes from that section. Review these outcomes before reading the section to help guide you through what is most relevant for you to take away from the material. This will also provide a brief look into what is to follow in that section.

Read the text. Read carefully the text to understand the problems and techniques. We will try to provide a plethora of examples, therefore, depending on your understanding of a topic, you many need to go carefully over all of the examples.

Explore the resources. Lastly, we recognize that there are many alternative methods of learning given the massive amounts of information and resources online. Thus, at the end of each section, there will be a number of superb resources that available on the internet in different formats. There are other free textbooks, informational websites, and also number of fantastic videos posted to youtube. We encourage to explore the resources to get another perspective on the material or to hear/read it taught from a differnt point of view or in presentation style.

Outline of this book

This book is divided in to 4 Parts:

Part I Linear Programming,

Part II Integer Programming,

Part III Discrete Algorithms,

Part IV Nonlinear Programming.

There are also a number of chapters of background material in the Appendix.

The content of this book is designed to encompass 2-3 full semester courses in an industrial engineering department.

Work in progress

This book is still a work in progress, so please feel free to send feedback, questions, comments, and edits to Robert Hildebrand at open.optimization@gmail.com.

1. Resources and Notation

Here are a list of resources that may be useful as alternative references or additional references.

FREE NOTES AND TEXTBOOKS

- [Mathematical Programming with Julia by Richard Lusby & Thomas Stidsen](#)
- [Linear Programming by K.J. Mtetwa, David](#)
- [A first course in optimization by Jon Lee](#)
- [Introduction to Optimizaiton Notes by Komei Fukuda](#)
- [Convex Optimization by Bord and Vandenberghe](#)
- [LP notes of Michel Goemans from MIT](#)
- [Understanding and Using Linear Programming - Matousek and Gärtner](#) [Downloadable from Springer with University account]
- [Operations Research Problems Statements and Solutions - Raúl PolerJosefa Mula Manuel Díaz-Madroñero](#) [Downloadable from Springer with University account]

NOTES, BOOKS, AND VIDEOS BY VARIOUS SOLVER GROUPS

- [AIMMS Optimization Modeling](#)
- [Optimization Modeling with LINGO by Linus Schrage](#)
- [The AMPL Book](#)
- [Microsoft Excel 2019 Data Analysis and Business Modeling, Sixth Edition, by Wayne Winston](#) - Available to read for free as an e-book through Virginia Tech library at Orielly.com.
- [Lesson files for the Winston Book](#)
- [Video instructions for solver and an example workbook](#)
- [youtube-OR-course](#)

GUROBI LINKS

- Go to <https://github.com/Gurobi> and download the example files.
- [Essential ingredients](#)
- [Gurobi Linear Programming tutorial](#)
- [Gurobi tutorial MILP](#)
- [GUROBI - Python 1 - Modeling with GUROBI in Python](#)
- [GUROBI - Python II: Advanced Algebraic Modeling with Python and Gurobi](#)
- [GUROBI - Python III: Optimization and Heuristics](#)
- [Webinar Materials](#)
- [GUROBI Tutorials](#)

HOW TO PROVE THINGS

- [Hammack - Book of Proof](#)

STATISTICS

- [Open Stax - Introductory Statistics](#)

LINEAR ALGEBRA

- [Beezer - A first course in linear algebra](#)
- [Selinger - Linear Algebra](#)
- [Cherney, Denton, Thomas, Waldron - Linear Algebra](#)

REAL ANALYSIS

- [Mathematical Analysis I by Elias Zakon](#)

DISCRETE MATHEMATICS, GRAPHS, ALGORITHMS, AND COMBINATORICS

- [Levin - Discrete Mathematics - An Open Introduction, 3rd edition](#)
- [Github - Discrete Mathematics: an Open Introduction](#) CC BY SA
- [Keller, Trotter - Applied Combinatorics](#) (CC-BY-SA 4.0)
- [Keller - Github - Applied Combinatorics](#)

MIXED INTEGER NONLINEAR PROGRAMMING

- [Mixed-Integer Nonlinear Optimization](#) Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, Jim Luedtke, and Ashutosh Mahajan

PROGRAMMING WITH PYTHON

- [A Byte of Python](#)
- [Github - Byte of Python](#) (CC-BY-SA)

Also, go to <https://github.com/open-optimization/open-optimization-or-examples> to look at more examples.

Notation

- $\mathbf{1}$ - a vector of all ones (the size of the vector depends on context)
- $\forall$ - for all
- $\exists$ - there exists
- $\in$ - in
- $\therefore$ - therefore
- $\Rightarrow$ - implies
- s.t. - such that (or sometimes "subject to".... from context?)
- $\{0, 1\}$ - the set of numbers 0 and 1
- $\mathbb{Z}$ - the set of integers (e.g. $1, 2, 3, -1, -2, -3, \dots$)
- $\mathbb{Q}$ - the set of rational numbers (numbers that can be written as p/q for $p, q \in \mathbb{Z}$ (e.g. $1, 1/6, 27/2$))
- $\mathbb{R}$ - the set of all real numbers (e.g. $1, 1.5, \pi, e, -11/5$)
- $\setminus$ - setminus, (e.g. $\{0, 1, 2, 3\} \setminus \{0, 3\} = \{1, 2\}$)
- $\cup$ - union (e.g. $\{1, 2\} \cup \{3, 5\} = \{1, 2, 3, 5\}$)

- $\cap$ - intersection (e.g. $\{1, 2, 3, 4\} \cap \{3, 4, 5, 6\} = \{3, 4\}$)
- $\{0, 1\}^4$ - the set of 4 dimensional vectors taking values 0 or 1, (e.g. $[0, 0, 1, 0]$ or $[1, 1, 1, 1]$)
- $\mathbb{Z}^4$ - the set of 4 dimensional vectors taking integer values (e.g., $[1, -5, 17, 3]$ or $[6, 2, -3, -11]$)
- $\mathbb{Q}^4$ - the set of 4 dimensional vectors taking rational values (e.g. $[1.5, 3.4, -2.4, 2]$)
- $\mathbb{R}^4$ - the set of 4 dimensional vectors taking real values (e.g. $[3, \pi, -e, \sqrt{2}]$)
- $\sum_{i=1}^4 i = 1 + 2 + 3 + 4$
- $\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2$
- $\sum_{i=1}^4 x_i = x_1 + x_2 + x_3 + x_4$
- $\square$ - this is a typical Q.E.D. symbol that you put at the end of a proof meaning "I proved it."
- For $x, y \in \mathbb{R}^3$, the following are equivalent (note, in other contexts, these notations can mean different things)

– $x^\top y$ *matrix multiplication*

– $x \cdot y$ *dot product*

– $\langle x, y \rangle$ *inner product*

and evaluate to $\sum_{i=1}^3 x_i y_i = x_1 y_1 + x_2 y_2 + x_3 y_3$.

A sample sentence:

$$\forall x \in \mathbb{Q}^n \exists y \in \mathbb{Z}^n \setminus \{0\}^n \text{ s.t. } x^\top y \in \{0, 1\}$$

"For all non-zero rational vectors x in n -dimensions, there exists a non-zero n -dimensional integer vector y such that the dot product of x with y evaluates to either 0 or 1."

2. Mathematical Programming

Outcomes

- *Consider an introductory problem*
- *Identify reasons for studying operations research*
- *Define "Mathematical Programming"*
- *Learn about different applications of the tools in this book*
- *Explore the different types of optimization models and what types we will see in this book*

2.1 Warm-Up Scenario

Imagine you're starting a new clothing company and decide to sell shirts. You've opened a store in Roanoke and, based on market research, you estimate the demand for the upcoming weekend:

- **Friday:** 5 shirts
- **Saturday:** 10 shirts
- **Sunday:** 7 shirts

Your supplier can deliver shirts in one day, meaning an order placed on Thursday will arrive on Friday, and so on. As a business owner, you need to decide how many shirts to order and when to place each order to ensure you meet the daily demand without overstocking.

*Defining the Problem To tackle this challenge, you'll need to think strategically about two main factors:

- **Ordering Quantities:** How many shirts to order on each day.
- **Inventory Levels:** How many shirts to keep in stock at the end of each day.

You can define these decisions using variables:

- Let the number of shirts you order on a specific day be a decision variable. For example, you could track the number of shirts ordered on Thursday, Friday, and Saturday.
- Define variables to represent the number of shirts you have in stock at the end of each day.

Your goal is to meet the projected demand while managing costs. Costs could include:

- The expense of ordering shirts.

- The cost of holding inventory.
- Penalties for running out of stock.

Refining the Scenario

As your business grows, additional complexities may arise. For instance:

- The cost of ordering shirts might vary by day due to supplier pricing.
- Holding inventory could incur storage costs.
- A budget might limit how much you can spend on orders.
- You might expand your product line to include pants and socks, each with its own demand and costs.

By clearly defining variables and constraints for these decisions, you can systematically approach the problem and develop an optimal strategy. This step-by-step process of defining the problem, incorporating real-world details, and refining the solution is a fundamental aspect of optimization.

Model Development Cycle

This multi-stage example illustrates the iterative cycle of modeling:

1. **Initial Problem:** Determine ordering quantities to meet demand.
2. **Modeling:** Formulate an optimization model based on available information.
3. **Solution:** Solve the model to find the optimal ordering plan.
4. **New Information:** Incorporate additional real-world complexities (varying costs, inventory costs, budget constraints, additional products, fixed ordering costs).
5. **Refinement:** Update the model to reflect new information and solve again.

Further Extensions

Looking ahead, there are several ways to expand this scenario:

- **Expanding to New Stores:** Opening additional stores introduces location-based demand, transportation costs, and coordination of inventory across multiple sites.
- **Hiring New Staff:** Incorporate labor costs, scheduling constraints, and productivity rates into the model.
- **Producing Clothing In-House:** Transitioning to manufacturing involves modeling production processes, including purchasing raw materials, machine capacities, and processing times.
- **Uncertain Demand:** In reality, demand may be uncertain. Advanced models can incorporate stochastic elements or use probabilistic methods to handle randomness.

In this course, we will focus on deterministic models where all data is known and fixed. Building a strong foundation with these models will prepare us to tackle more complex problems involving uncertainty and randomness in future studies.

2.2 Why Study Operations Research?

Operations Research (OR) is a discipline that applies advanced analytical methods to help make better decisions. In a world where resources are limited and organizations face complex challenges, OR provides the tools and methodologies to model, analyze, and solve problems involving the optimal allocation of scarce resources. Studying OR equips individuals with a systematic and quantitative approach to decision-making, which is invaluable in industries such as logistics, finance, healthcare, and manufacturing.

By understanding OR, you gain the ability to construct mathematical models of real-world systems, allowing for the simulation and optimization of different scenarios. This leads to improved efficiency, reduced costs, and enhanced performance. Moreover, the skills developed through studying OR—such as critical thinking, problem-solving, and data analysis—are highly transferable and sought after in today's data-driven economy.

Operations Research also bridges the gap between theory and practice. It transforms abstract mathematical concepts into actionable strategies that can have a tangible impact on organizational success. Whether it's optimizing supply chains, scheduling flights, or managing portfolios, the applications of OR are vast and impactful.

2.3 What is Mathematical Programming?

Mathematical Programming is a core area within Operations Research that focuses on the formulation and solution of optimization problems. It involves creating mathematical models that represent the decision-making process of a system, where the goal is to find the best possible outcome under given constraints. The term "programming" in this context refers to planning or scheduling, not computer programming.

At the heart of Mathematical Programming is the objective function—a mathematical expression that defines the criterion to be optimized, such as minimizing costs or maximizing profits. The decision variables represent the choices available, and the constraints reflect the limitations or requirements of the problem, such as resource capacities or regulatory conditions.

There are various types of mathematical programming, each suited to different kinds of problems:

- **Linear Programming (LP):** Deals with problems where both the objective function and constraints are linear.
- **Integer Programming (IP):** Involves decision variables that must take on integer values.

- **Nonlinear Programming (NLP):** Handles problems with nonlinear objective functions or constraints.
- **Dynamic Programming (DP):** Breaks down problems into simpler subproblems and is particularly useful for multistage decision processes.

Mathematical Programming provides a powerful framework for optimizing complex systems and making informed decisions. It combines elements of mathematics, economics, and computer science to develop solutions that are both efficient and effective.

2.4 Applications

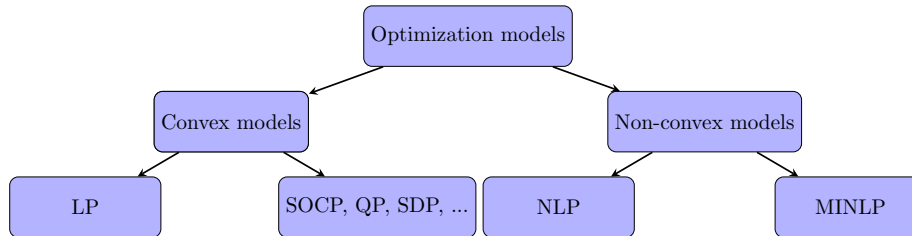
Operations Research and Mathematical Programming have a wide range of applications across various industries and sectors:

- **Transportation and Logistics:** Optimizing routing and scheduling for delivery vehicles, reducing transportation costs, and improving supply chain efficiency.
- **Manufacturing:** Streamlining production processes, managing inventory levels, and scheduling machinery to maximize productivity and minimize waste.
- **Finance:** Portfolio optimization, risk assessment, and capital budgeting to enhance financial performance and manage uncertainties.
- **Healthcare:** Allocating resources such as staff and equipment, scheduling surgeries, and managing patient flow to improve service quality and reduce wait times.
- **Energy Sector:** Optimizing power generation and distribution, planning for renewable energy integration, and managing energy consumption.
- **Telecommunications:** Network design and optimization, bandwidth allocation, and improving the reliability and efficiency of communication systems.
- **Agriculture:** Planning crop rotations, optimizing the use of fertilizers and water, and managing supply chains for agricultural products.
- **Public Sector:** Urban planning, emergency response logistics, and policy analysis to enhance public services and resource utilization.

These applications demonstrate the versatility and impact of OR and Mathematical Programming. By leveraging these tools, organizations can make data-driven decisions that lead to significant improvements in performance, cost savings, and competitive advantage. As global challenges become more complex, the importance of skilled professionals in Operations Research continues to grow.

2.5 Types of Optimization problems

We will state main general problem classes to be associated with in these notes. These are Linear Programming (LP), Mixed-Integer Linear Programming (MILP), Non-Linear Programming (NLP), and Mixed-Integer Non-Linear Programming (MINLP).



Along with each problem class, we will associate a complexity class for the general version of the problem. Complexity classes are discussed in a later portion of the book. Although we will often state that input data for a problem comes from $\mathbb{R}$ (as a real number), when we discuss complexity of such a problem, we actually mean that the data is rational, i.e., from $\mathbb{Q}$ (rational numbers that are encoded in the computer typically), and is given in binary encoding.

CAUTION!

In the following subsections, we will use advanced notation that may be unfamiliar at this point. These mathematical notations will be developed throughout the book.

Furthermore, don't be too concerned about the specific definitions of the complexity classes at this point. For now, you can read that *Polynomial time (P)* means it should be relatively easy to solve, while the other classes might be much more difficult to solve.

2.6 Linear Programming (LP)

Some linear programming background, theory, and examples will be provided in ??.

Linear Programming (LP):

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem is

$$\begin{aligned}
 \max \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \geq 0
 \end{aligned} \tag{2.1}$$

Linear programming can come in several forms, whether we are maximizing or minimizing, or if the constraints are $\leq$, $=$ or $\geq$. One form commonly used is *Standard Form* given as

Linear Programming (LP) Standard Form:

Polynomial time (P)

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *linear programming* problem in *standard form* is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax = b \\ & x \geq 0 \end{aligned} \tag{2.2}$$

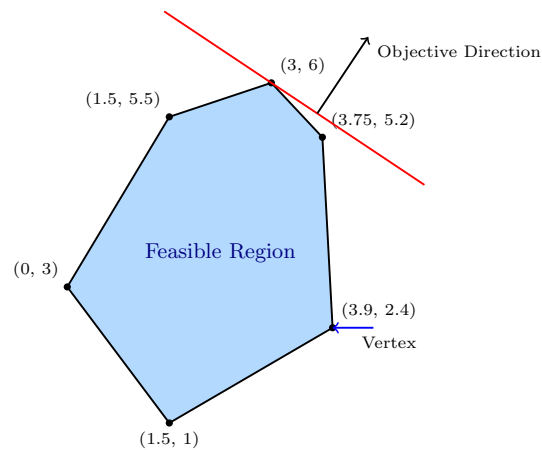


Figure 2.1: Linear programming constraints and objective.

2.7 Mixed-Integer Linear Programming (MILP)

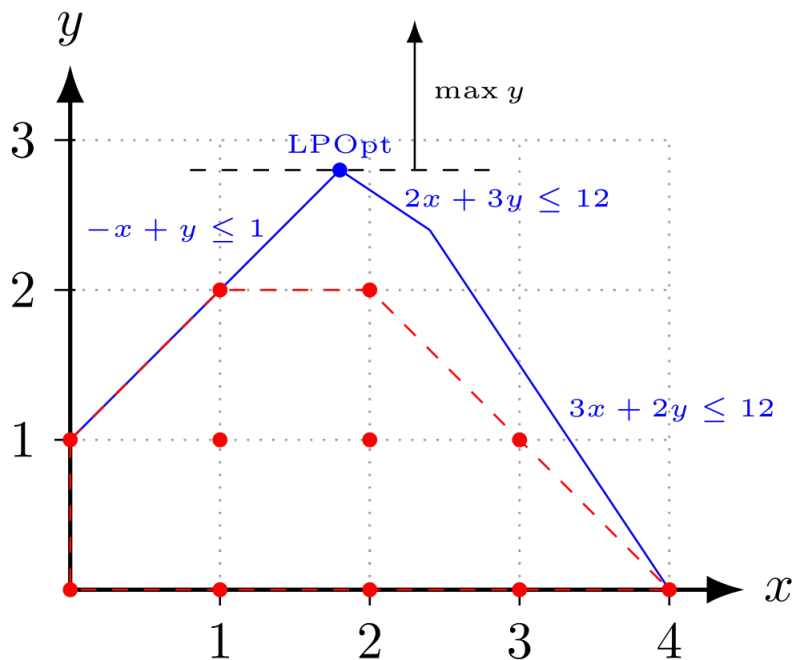
Mixed-integer linear programming will be the focus of Sections 13, 15, 14, and ?? . Recall that the notation $\mathbb{Z}$ means the set of integers and the set $\mathbb{R}$ means the set of real numbers. The first problem of interest here is a *binary integer program* (BIP) where all n variables are binary (either 0 or 1).

Binary Integer programming (BIP):*NP-Complete*

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \{0, 1\}^n \end{aligned} \tag{2.1}$$

A slightly more general class is the class of *Integer Linear Programs* (ILP). Often this is referred to as *Integer Program* (IP), although this term could leave open the possibility of non-linear parts.



© Fanosta [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/)¹

Figure 2.2: Comparing the LP relaxation to the IP solutions.

Figure 2.2

Integer Linear Programming (ILP):*NP-Complete*

¹IP polytope with LP relaxation, from https://en.wikipedia.org/wiki/Integer_programming#/media/File:IP_polytope_with_LP_relaxation.svg, Fanosta [CC BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/), 2019.

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \end{aligned} \tag{2.2}$$

An even more general class is *Mixed-Integer Linear Programming (MILP)*. This is where we have n integer variables $x_1, \dots, x_n \in \mathbb{Z}$ and d continuous variables $x_{n+1}, \dots, x_{n+d} \in \mathbb{R}$. Succinctly, we can write this as $x \in \mathbb{Z}^n \times \mathbb{R}^d$, where $\times$ stands for the *cross-product* between two spaces.

Below, the matrix A now has $n + d$ columns, that is, $A \in \mathbb{R}^{m \times (n+d)}$. Also note that we have not explicitly enforced non-negativity on the variables. If there are non-negativity restrictions, this can be assumed to be a part of the inequality description $Ax \leq b$.

Mixed-Integer Linear Programming (MILP):

NP-Complete

Given a matrix $A \in \mathbb{R}^{m \times (n+d)}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^{n+d}$, the *mixed-integer linear programming* problem is

$$\begin{aligned} \max \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b \\ & x \in \mathbb{Z}^n \times \mathbb{R}^d \end{aligned} \tag{2.3}$$

2.8 Non-Linear Programming (NLP)

NLP:

NP-Hard

Given a function $f(x): \mathbb{R}^d \rightarrow \mathbb{R}$ and other functions $f_i(x): \mathbb{R}^d \rightarrow \mathbb{R}$ for $i = 1, \dots, m$, the *nonlinear programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1, \dots, m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.1}$$

Nonlinear programming can be separated into convex programming and non-convex programming. These two are very different beasts and it is important to distinguish between the two.

2.8.1. Convex Programming

Here the functions are all **convex**!

Convex Programming:

Polynomial time (P) (typically)

Given a convex function $f(x): \mathbb{R}^d \rightarrow \mathbb{R}$ and convex functions $f_i(x): \mathbb{R}^d \rightarrow \mathbb{R}$ for $i = 1, \dots, m$, the *convex programming* problem is

$$\begin{aligned} \min \quad & f(x) \\ \text{s.t.} \quad & f_i(x) \leq 0 \quad \text{for } i = 1, \dots, m \\ & x \in \mathbb{R}^d \end{aligned} \tag{2.2}$$

Observe that convex programming is a generalization of linear programming. This can be seen by letting $f(x) = c^\top x$ and $f_i(x) = A_i x - b_i$.

2.8.2. Non-Convex Non-linear Programming

When the function f or functions f_i are non-convex, this becomes a non-convex nonlinear programming problem. There are a few complexity issues with this.

IP AS NLP As seen above, quadratic constraints can be used to create a feasible region with discrete solutions. For example

$$x(1 - x) = 0$$

has exactly two solutions: $x = 0, x = 1$. Thus, quadratic constraints can be used to model binary constraints.

Binary Integer programming (BIP) as a NLP:

NP-Hard

Given a matrix $A \in \mathbb{R}^{m \times n}$, vector $b \in \mathbb{R}^m$ and vector $c \in \mathbb{R}^n$, the *binary integer programming* problem

is

$$\begin{aligned}
 \max \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & \cancel{x \in \{0,1\}^n} \\
 & x_i(1-x_i) = 0 \quad \text{for } i = 1, \dots, n
 \end{aligned} \tag{2.3}$$

Alternatively, consider the transformation where $x_i \in \{-1, 1\}$.

$$\begin{aligned}
 \min \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & x \in \{-1, 1\}^n
 \end{aligned}$$

This can be reformulated with a single nonconvex constraint as

$$\begin{aligned}
 \min \quad & c^\top x \\
 \text{s.t.} \quad & Ax \leq b \\
 & -1 \leq x_j \leq 1, \quad 1 \leq j \leq n, \\
 & \|x\|^2 \geq n.
 \end{aligned}$$

2.8.3. Machine Learning

Machine learning problems are often cast as continuous optimization problems, which involve adjusting parameters to minimize or maximize a particular objective. Frequently they are convex optimization problems, but many turn out to be nonconvex. Here are two examples of how these problems arise at a glance. We will see examples in greater detail later in the book.

Loss Function Minimization

In supervised learning, this objective is typically a loss function L that quantifies the discrepancy between the predictions of a model and the true data labels. The aim is to adjust the parameters θ of the model to minimize this loss. Mathematically, this can be represented as:

$$\min_{\theta} L(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i; \theta)) \tag{2.4}$$

where N is the number of data points, l is a per-data-point loss (e.g., squared error for regression or cross-entropy for classification), y_i is the true label for the i -th data point, and $f(x_i; \theta)$ is the model's prediction for the i -th data point with parameters θ .

Clustering Formulation

Clustering, on the other hand, seeks to group or partition data points such that data points in the same group are more similar to each other than those in other groups. One popular method is the k-means clustering algorithm. The objective of k-means is to partition the data into k clusters by minimizing the within-cluster sum of squares (WCSS). The mathematical formulation can be given as:

$$\min_{\mathbf{c}_1, \dots, \mathbf{c}_k} \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mathbf{c}_j\|^2 \quad (2.5)$$

where C_j represents the j -th cluster and $\mathbf{c}_j$ is the centroid of that cluster.

This encapsulation presents a glimpse into how ML problems are framed mathematically. In practice, numerous algorithms, constraints, and regularizations add complexity to these basic formulations.

2.9 Mixed Integer Non-Linear Programming (MINLP)

MINLP:

NP-Hard

Mixed Integer Nonlinear Programming (MINLP) combines elements of integer programming and non-linear programming. In MINLP, some or all of the decision variables are constrained to be integers, and the objective function or constraints are nonlinear. A general MINLP problem can be formulated as:

$$\begin{aligned} \min \quad & f(x, y) \\ \text{s.t.} \quad & g_i(x, y) \leq 0 \quad \text{for } i = 1, \dots, m \\ & h_j(x, y) = 0 \quad \text{for } j = 1, \dots, p \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \quad (2.1)$$

where f is the objective function, g_i and h_j are constraint functions, x is a vector of continuous variables, and y is a vector of integer variables.

MINLP is particularly challenging due to the nonlinearity in the objective and/or constraints and the discrete nature of some decision variables. It finds applications in various fields such as industry for process optimization, computational geometry, and machine learning for hyperparameter tuning.

2.9.1. Convex MINLP

In the convex case, both the objective function and the constraints are convex functions. This subclass is easier to solve compared to its non-convex counterpart.

Convex MINLP:

NP-Hard, but *Polynomial time (P)* in fixed dimension (typically)

For a convex MINLP, the problem is defined as:

$$\begin{aligned} \min \quad & f(x,y) \quad (\text{convex}) \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad (\text{convex constraints}) \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.2}$$

Convex MINLPs, while still challenging, are typically more tractable due to the properties of convexity, which allow for more efficient solution methods.

2.9.2. Non-Convex MINLP

Non-convex MINLPs are significantly harder due to the presence of non-convex functions, which can lead to multiple local minima.

Non-Convex MINLP:

NP-Hard (in fact, undecidable)

The non-convex MINLP problem is formulated as:

$$\begin{aligned} \min \quad & f(x,y) \quad (\text{non-convex}) \\ \text{s.t.} \quad & g_i(x,y) \leq 0 \quad (\text{possibly non-convex constraints}) \\ & x \in \mathbb{R}^n, y \in \mathbb{Z}^k \end{aligned} \tag{2.3}$$

Non-convex MINLPs pose significant computational challenges due to the possibility of multiple local optima and the inherent complexity of integer constraints. These problems are common in real-world applications where decisions are discrete, and the system behavior is non-linear and complex.

Complexity and Applications

MINLP problems are known for their computational complexity, primarily due to the combination of non-linearity and integrality. The non-convex variants, in particular, are *NP-Hard*, making them some of the most challenging problems in optimization.

In practice, MINLP models find extensive applications across various domains. In industry, they are used for complex decision-making processes like supply chain optimization and production planning. In computational geometry, MINLP techniques help in solving problems like optimal packing or layout design. Furthermore, in the realm of machine learning, MINLPs are crucial for tasks like feature selection and hyperparameter optimization where discrete choices and nonlinear relationships are involved.

The versatility of MINLP models, combined with their inherent complexity, makes them a fascinating and essential area of study in the field of optimization.

Category	Software (License)
Linear programming (free)	CDD, CLP, GLPK, LPSOLVE, OSQP, SCIP
Mixed Integer Linear programming (free)	CBC, GLPK, LPSOLVE, SCIP
Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, LINPROG, MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Linear programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), INTLINPROG, KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Quadratic programming (free)	OSQP, CLP, DAQP (mixed-binary), OOQP, QPC, QPOASES, QUADPROGGBB (nonconvex QP)
Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), QUADPROG, XPRESS (free for academia)
Mixed Integer Quadratic programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), KNITRO, MOSEK (free for academia), XPRESS (free for academia)
Second-order cone programming (free)	ECOS, SCS, SDPT3, SEDUMI
Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), CONEPROG, GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Mixed Integer Second-order cone programming (commercial)	COPT (free for academia), CPLEX (free for academia), GUROBI (free for academia), MOSEK (free for academia), XPRESS (free for academia)
Semidefinite programming (free)	CSDP, DSDP, LOGDETTPPA, PENLAB, SCS, SDPA, SDPLR, SDPT3, SDPNAL, SEDUMI
Semidefinite programming (commercial)	COPT (free for academia), LMILab (not recommended), MOSEK (free for academia), PENBMI, PENSDP (free for academia)
General nonlinear programming and other solvers	BARON, FILTERSD, FMINCON, GPPSOY, IPOPT, KNITRO, LMIRank, MPT, NOMAD, PENLAB, SNOPT, SPARSEPOP

Table 2.1: Optimization Software and their Licenses

2.10 Optimization Under Uncertainty

Optimization problems often involve uncertainty in their parameters, whether due to measurement errors, variability in input data, or unpredictable future events. Addressing uncertainty is crucial for developing solutions that are effective in real-world settings. Two primary approaches to optimization under uncertainty are stochastic optimization and robust optimization.

2.10.1. Stochastic Optimization

Stochastic optimization deals with problems where some parameters are uncertain but can be described probabilistically. In this framework, the optimization process incorporates the expected values, variances, or other statistical characteristics of uncertain parameters. Solutions are often evaluated based on their performance over a distribution of possible scenarios.

For example, in supply chain management, uncertain demand can be modeled using probability distributions. A company may aim to minimize costs while ensuring a high service level under varying demand conditions. Similarly, in finance, portfolio optimization often accounts for uncertain future returns by maximizing expected returns while managing risk.

2.10.2. Robust Optimization

Robust optimization, on the other hand, focuses on creating solutions that perform well across the worst-case realizations of uncertainty. Rather than assuming a specific probability distribution, robust optimization works with uncertainty sets, which define the range of possible values that uncertain parameters can take. The goal is to find solutions that are feasible and effective for all possible scenarios within the uncertainty set.

Applications of robust optimization include power grid operations, where uncertainties in demand and renewable energy generation need to be managed, and scheduling problems, such as airline crew scheduling, where disruptions must be minimized under varying operational conditions.

2.10.3. Applications

Optimization under uncertainty has a wide array of applications across industries:

- **Healthcare:** Designing vaccination strategies to minimize the spread of diseases while accounting for uncertain infection rates and patient behaviors.
- **Transportation:** Managing traffic flow or public transit schedules under uncertain demand and travel times.
- **Energy:** Planning renewable energy investments and operations under uncertain weather conditions and energy prices.

- **Manufacturing:** Ensuring production schedules are resilient to supply chain disruptions and variability in raw material quality.

Addressing the complexity of uncertain data typically makes these problems much harder than their deterministic counterparts. While these methods are powerful and applicable to a wide range of real-world scenarios, we will not focus on optimization under uncertainty in this textbook.